The Evolution of Computing Paradigms

A Comprehensive History of Programming Languages

Wojciech Fiołka, Paweł Kojma, Michał Bednarz, Jakub Chomiczewski, Filip Rabiega

Department of Computer Science

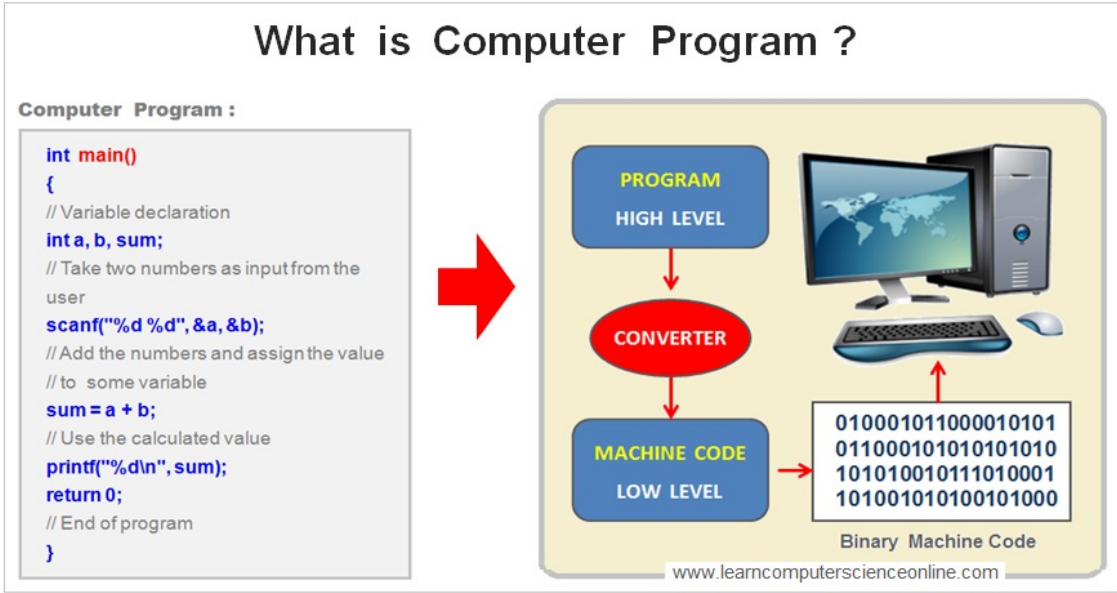
March 29, 2026

What is a Programming Language?

A programming language is a structured way for humans to communicate instructions to a machine. Computers fundamentally operate on binary logic, which is unintuitive for humans.

They are essential because they:

- Translate human intent into machine-executable steps
- Manage complexity through abstraction
- Enable scalable and maintainable systems



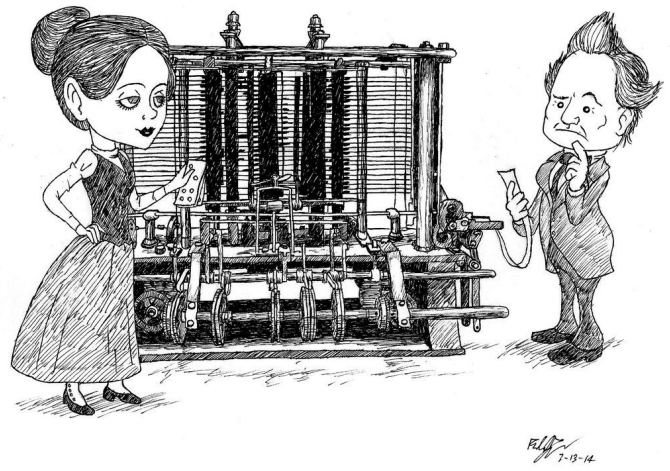
Early Concept of an Algorithm

Long before modern computers, the concept of an algorithm existed as a precise sequence of steps to solve a problem.

In the 19th century:

- Charles Babbage designed the Analytical Engine
- Ada Lovelace described symbolic computation
- First algorithm intended for machine execution

This marked the conceptual birth of programming.

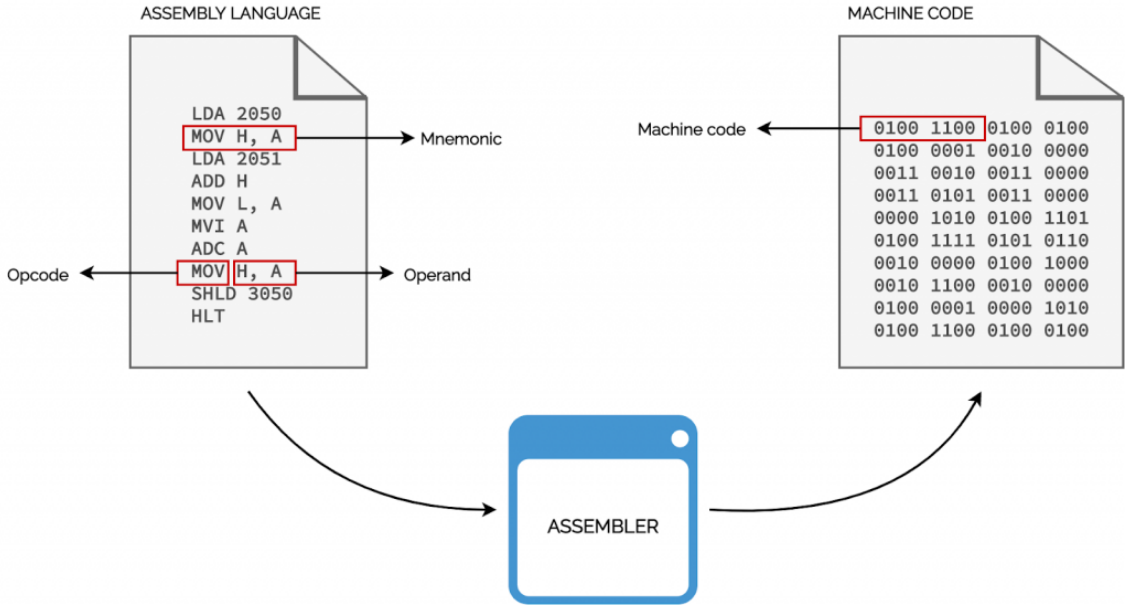


Assembly Language

Assembly language was the first step away from raw machine code, replacing binary with mnemonic instructions.

Key characteristics:

- Very close to hardware
- Nearly 1:1 with CPU instructions
- Efficient but difficult to maintain
- Architecture-dependent



Fortran

Fortran (FORmula TRANslator) was one of the first high-level programming languages, created for scientific computation. Developed by John Backus at IBM

It introduced:

- Mathematical expressions in code
- Compiler abstraction
- Efficient numerical processing

COBOL

COBOL was designed for business and administrative applications. It is an imperative, procedural, object-oriented language. Its main ideas:

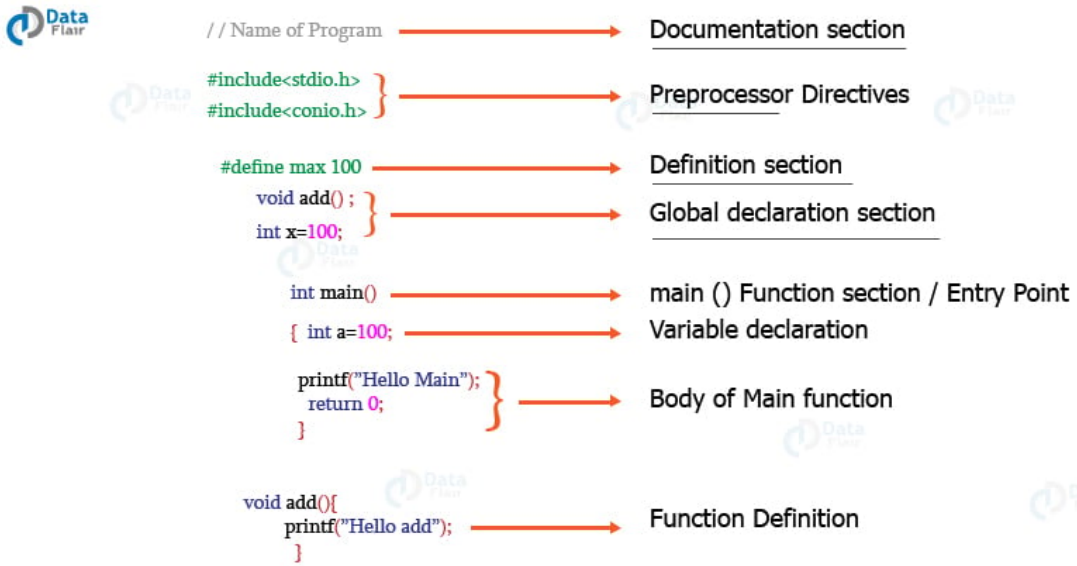
- English-like syntax
- Focus on data processing
- Widely used in finance and government

C Language

The C language balances low-level control with high-level abstraction.

It enabled:

- Direct memory manipulation
- Portable system programming
- Efficient development

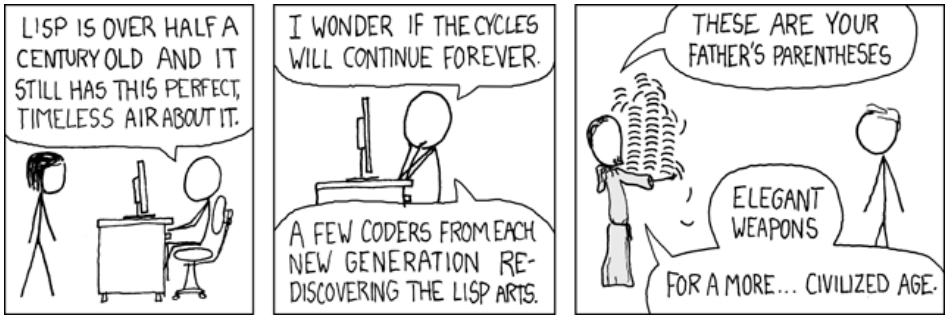


LISP

The LISP family of programming languages is one of the oldest high-level language families, created in the late 1950s. Notable dialects include Scheme, Common Lisp, and Clojure.

Key characteristics:

- Based on symbolic computation and list processing
- Uses a simple and uniform syntax (prefix notation)
- Code and data share the same structure (homoiconicity)
- Widely used in artificial intelligence research



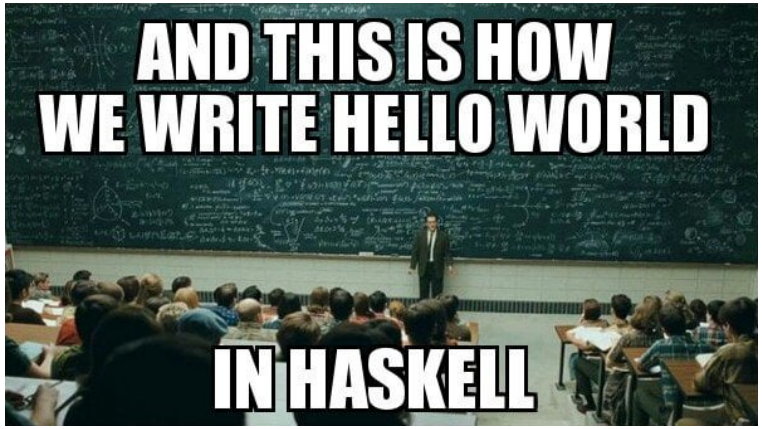
Structure and logic

Spaghetti code

To combat unmanageable "spaghetti code," the industry embraced modularity and formal flow control. This era replaced arbitrary jumps with nested logic and structured blocks. It proved that programming could be a disciplined engineering craft, focused on readable, verifiable execution paths.

Haskell

It was designed to unify the functional programming community, offering a purely functional approach while advancing sophisticated type systems. It also introduced lazy evaluation, a feature that was quite unusual at the time.



The joys of purely functional programming

Object oriented shift

Complexity management shifted from procedural steps to data-centric encapsulation. By bundling state and behavior into objects, developers gained a new way to build and maintain massive systems. This paradigm shift redefined how software is architected, focusing on modeling real-world entities.

Object oriented design patterns

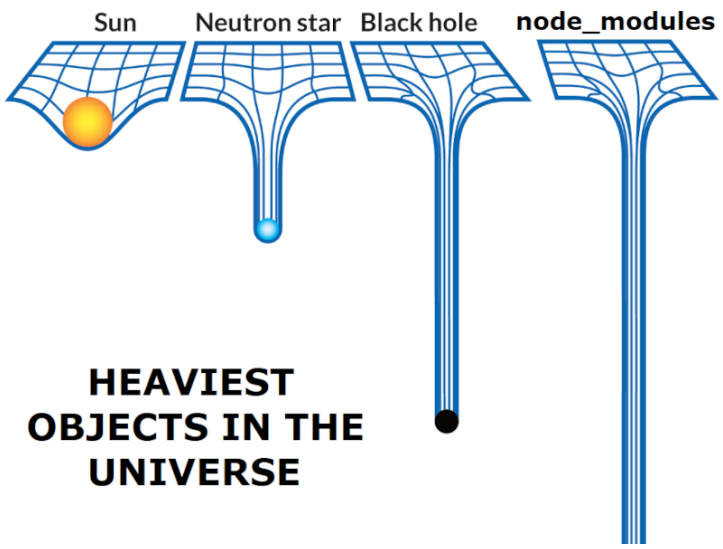
OCaml

funni comment

It was designed to extend the ML family by combining functional programming with practical features for real-world use. It offers a strong static type system with powerful type inference, while also supporting an object-oriented style, making it both functional and object-oriented. This balance allows it to be expressive and efficient, suitable for both research and industrial applications.

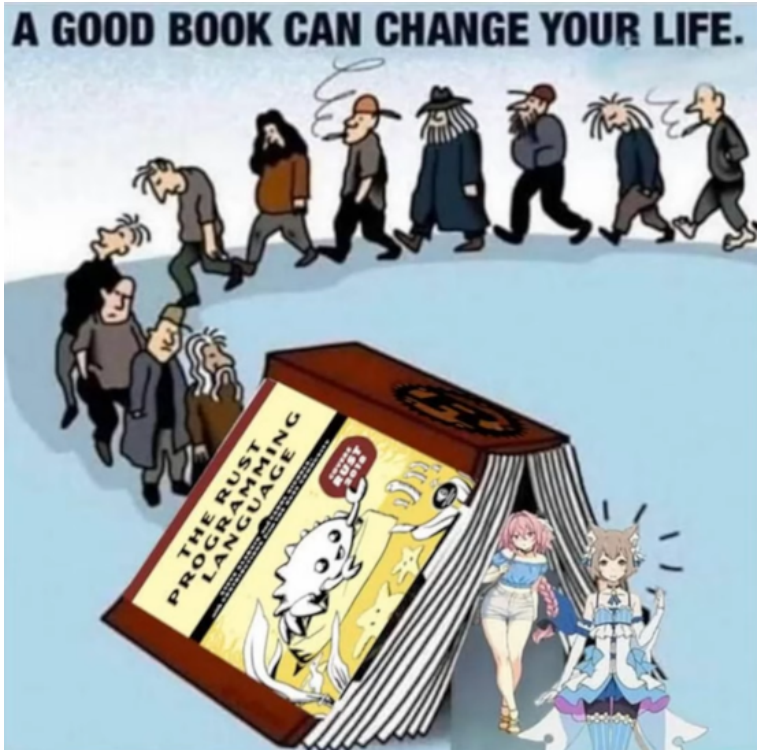
Scripting and productivity boom

As the internet scaled, the focus shifted from machine efficiency to "developer velocity." High-level scripting and virtual machines prioritized human time, allowing for rapid prototyping and platform independence. Programming became more about the speed of turning ideas into functional software than managing hardware constraints.



Node.js programming environment

Contemporary language trends



Path of a Rust programmer

Modern development focuses on "guardrails" eliminating memory errors and concurrency bugs at the compiler level. This is also an era of refined research; projects like Fram (UWr) are pushing the ML tradition forward by using algebraic effects and implicit parameters to handle side effects with modular, ergonomic precision.

Future of programming

Future of programming with LLMs

We are moving from "writing" code to "orchestrating" intent. The rise of AI agents shifts the programmer's role toward high-level architecture, while the dawn of quantum computing demands an entirely new logical framework, shifting our focus from bits and states to the complex probabilities of qubits and superposition.

Main sources

- Scriptol. *History and Evolution of Programming Languages*.
  `scriptol.com/programming/history.php`
- Praxent. *A History of Programming Languages: From Plankalkül to Swift*.
  `praxent.com/blog/history-of-programming-languages`
- van Dam, A. (Brown University). *History of Programming Languages Reference*.
  `cs.brown.edu/people/adf/programming_languages.html`
- Wikipedia. *History of Programming Languages*.
  `en.wikipedia.org/wiki/History_of_programming_languages`
- IEEE Computer Society. *The Evolution of Programming Languages Timeline*.
  `computer.org/publications/tech-news/...`